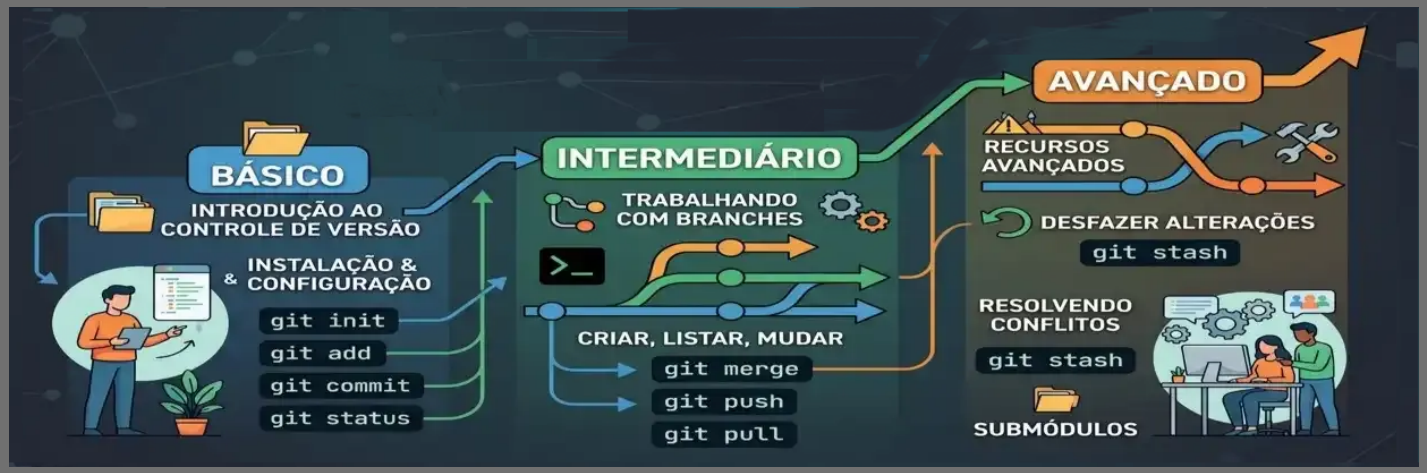
GIT

Agenda

- Git



Os comandos Git permitem a interação com o sistema de controle de versão para gerenciar repositórios e acompanhar alterações no código. Com eles criamos, modificamos e sincronizamos repositórios do ambiente local com os repositórios remotos, tornando o fluxo de trabalho mais controlado...

Git - Introdução

- Sistema de controle de versão de documentos/códigos
- É um sistema distribuído
- Muito utilizado por equipes de desenvolvimento de SW
- Criado em 2005 por **Linus Torvalds**, criador também do kernel do Linux
- O Git mantém um **histórico** completo **das alterações realizadas** nos arquivos de código
- Permite a criação de diversas ramificações (*branchs*) no projeto para trabalhar em *features* ou correções separadamente, podendo-se mesclar as ramificações...

Git - Introdução

Fluxo de Trabalho do Git x GitHub

GitHub
Flow

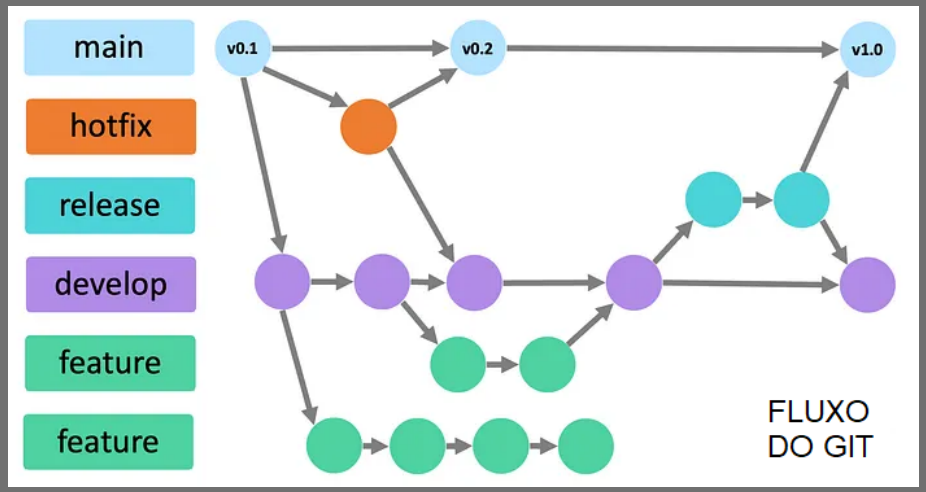
OR

Git
Flow

Git - Introdução

Fluxo de Trabalho do Git

- É um modelo de ramificação mais complexo que fornece uma abordagem estruturada para gerenciar grandes projetos que introduzem dois ramos de longa duração – ***develop*** e ***main*** – junto com vários ramos de suporte/apoio



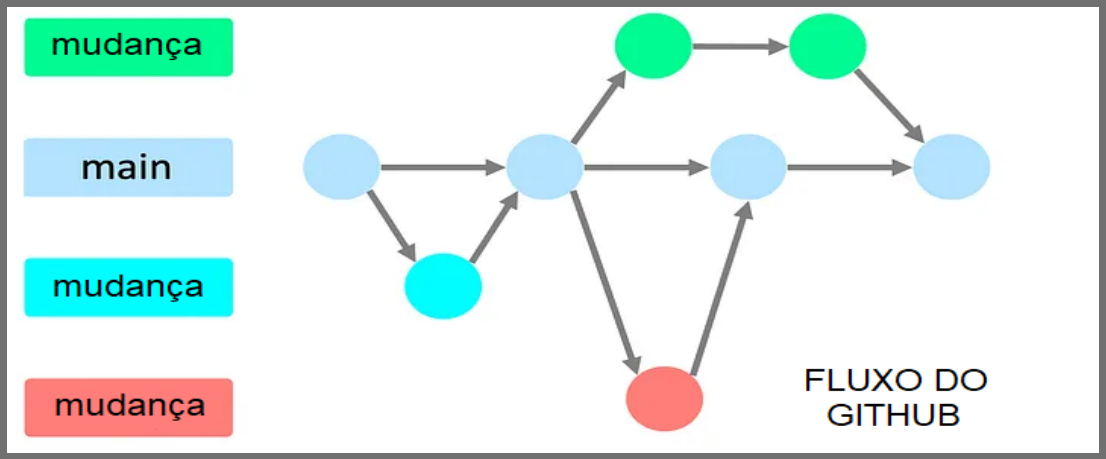
- Principais pontos
  - Ramo de **desenvolvimento**: serve como ramo de integração de recursos, contendo código em progresso
  - Ramos de **recursos**: são criados a partir do ramo de **desenvolvimento** para trabalhar em novos recursos ou correções de *bugs*
  - Ramos de **lançamento**: qdo um conj. de recursos está pronto para lançamento, um ramo de **lançamento** é criado a partir do ramo de **desenvolvimento** para testes finais e correções de bugs
  - Ramos de **correção**: são usadas para resolver rapidamente problemas críticos na base de código de produção (criados a partir do ramo **main**)

O **Fluxo do Git** fornece uma abordagem estruturada, tornando-o ideal para proj.s maiores com lançamentos agendados e múltiplas equipes de desenvolvimento...

Git - Introdução

Fluxo de Trabalho do GitHub

- Gira em torno do conceito de ramificações e solicitações de puxadas (*pull requests*), favorecendo equipes que trabalham em integração contínua e implantações frequentes



- Principais pontos:
  - Ramo **main**: sempre implantável, última versão estável do proj.
  - Ramo **feature**: criado para cada funcionalidade nova ou erro corrigido (ramo isolado do **main** até estar pronto para revisões)
  - **Pull Requests**: ao concluir o trabalho num ramo **feature**, o desenvolvedor faz uma solicitação de puxada para iniciar a revisão e discussão do código
  - Implantação contínua: depois que uma *pull request* é aprovada, as alterações são mescladas no ramo **main** e implantadas automaticamente na produção

O **Fluxo do GitHub** é adequado para proj.s com implantações frequentes e equipes pequenas, enfatizando simplicidade e integração contínua...

Git - Introdução

Fluxo de Trabalho do GitHub

1. **Crie uma ramificação**: crie um novo ramo a partir do ramo **main** com um nome descritivo (fix-header-style) para isolar as suas alterações
2. **Faça alterações e confirme**: edite arquivos e salve o progresso usando confirmações com mensagens claras. *Commits* regulares criam um histórico do seu trabalho
3. **Abra uma *pull request*** (PR): quando estiver pronto para receber *feedback* dos seus pares (ou mesmo enquanto o trabalho estiver em andamento), abra um PR para propor a fusão de seu *branch* com o **main**
4. **Revise e discuta**: os colaboradores revisam o código, deixam comentários e sugerem melhorias. Você pode continuar enviando *commits* para o mesmo *branch* para responder a esse *feedback*
5. **Mesclar**: assim que o PR for aprovado e passar nos testes automatizados, mescle-o no *branch* **main**. Isso integra suas alterações no código-base de produção
6. **Exclua a ramificação**: após a fusão, exclua a ramificação do recurso para manter o repositório limpo e indicar que o trabalho foi concluído

Git - Introdução

Fluxos de Trabalho do GitHub x Git

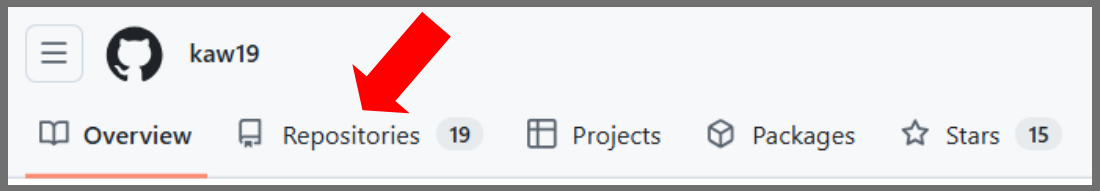
Fluxo GitHub x Fluxo Git		
Recurso	**Fluxo do GitHub**	**Fluxo do GitHub**
Ideia Principal	Simples, leve, para lançamentos rápidos	Estruturado, multifilial, para lançamentos agendados
Ramos principais	*main* e *features*	*main, develop, release, hotfix* e *feature*
Complexidade	Baixa; ideal para aplicativos web/SaaS	Alto; ideal para *software* versionado (por exemplo, aplicativos móveis)
Implantação	Contínua, geralmente várias vezes ao dia	Lançamentos periódicos e/ou versionados

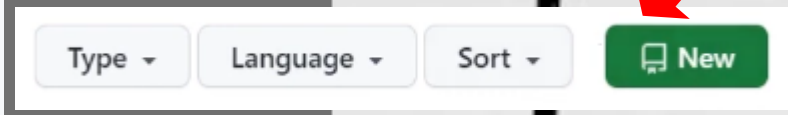
Git - Instalação

- Se não possuir conta no Github crie sua conta gratuita seguindo estes passos:
  - Acesse GitHub.com e clique em ***Sign up***
  - Escolha ***free acount***
  - Você receberá um *e-mail* de verificação no endereço fornecido
  - Clique no *link* para concluir o processo de verificação
- Instale o git no seu S.O. (máquina local)
  - Já tem o Git? Abra o **Terminal** se estiver usando um Mac ou o **PowerShell** se estiver usando um PC com Windows e digite:
    ```
    $ git version
      git version 2.33.1
    ```
  - Baixando e instalando o Git:
    - Baixe-o em www.git-scm.com
    - instale o Git conforme guia ProGit (http://git-scm.com/book/en/v2/Getting-Started-Installing-Git)

Git - Repositório no Github

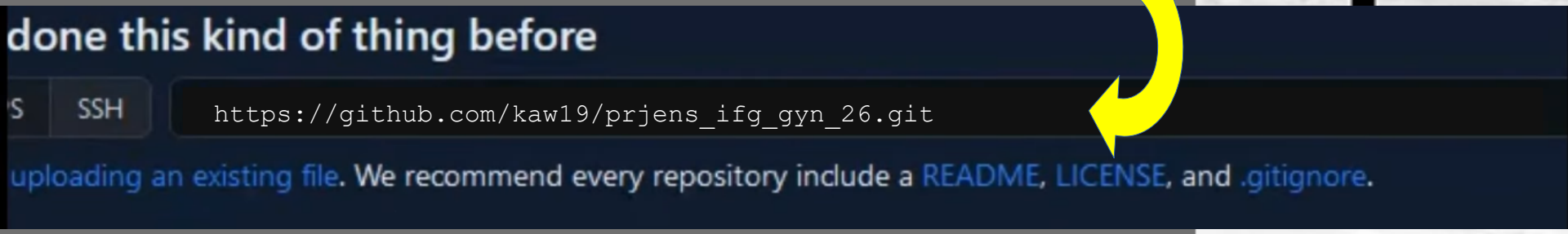
- Crie um **repositório** no **Github** (plataforma *online* que hospeda os repositórios controlados pelo **git**)
- Faça um *login* em sua conta no **Github**, acesse **Repositories** e clique em **New**





- Preencha os campos e no final da página clique em **Create repository**

- Anote o endereço do repositório criado no Github



Git - Comandos

- **Básicos**
  - Iniciar um novo repositório
  - Preparar alterações para um *commit* (confirmação)
  - Verificar o status dos arquivos modificados
  - Trabalhar com ramos/ramificações (*branches*)
  - Fazer mesclagens de ramos (*merge*)
  - Marcar versões específicas para facilitar a identificação
  - Lidar com repositórios remotos (*clone*)
- Avançados
  - *blame* e *bisect* ajudam a depurar problemas e rastrear a origem de erros no histórico do código

Git

Comandos Avançados

Comandos Intermediários

Comandos Básicos

```
init, add, commit, remote, status, push,
checkout, merge, log, rm, mv, diff, config
```

Git

Comandos Básicos

- Crie um repositório local na pasta do projeto que terá versões controladas:

```
c:\prj\ifg\ens\26> git init [nome_projeto]
```

- Clonando um repositório remoto:
  - Abra um tela com o terminal e digite:

```
c:..\> git clone https://github.com/githubschool/scratch.git
Cloning into 'scratch'...
remote: Counting objects: 6, done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (6/6), done.
```

Git

Comandos Básicos

- **Add**: Adiciona arquivo(s) ao repositório
- **Commit**: Anexa mensagem às alterações (histór.)

```
git add nome_do_arquivo

git commit -m "mensagem dessa confirmação de alterações"
```

- Informando o endereço do repositório remoto (normalmente no Github)

```
git remote add origin https://github.com/kaw19/prjens_ifg_gyn_26
```

Git

Comandos Básicos

- **Status**: Exibe informações sobre as modificações e o estado de preparação dos arq.s

```
git staus
```

- **Push**: Atualiza o ramo **etapa2** do repositório remoto a partir do repositório local:

```
git push -u origin etapa2
```

O comando **git push** envia *commits* locais do seu repositório Git para um repositório remoto (ex: GitHub, GitLab), atualizando o ramo (*branch*) remoto com seu histórico local. Ele sincroniza o código desenvolvido na máquina local com o servidor.

- `git push origin <branch-name>`: Envia o ramo atual para o repositório remoto '*origin*'.
- `git push -u origin <branch-name>`: Define o ramo remoto como padrão (*upstream*) para envios futuros.

Git

Comandos Básicos

- **Push**: Depois de alguma nova alteração realizada no código (arquivos locais), é preciso atualizar o repositório remoto:

```
git add nome_arquivo_alterado

git commit -m "mensagem essas alterações"

git push
```

O comando **checkout** informa ao Git qual **ramo** ou **commit** que e deseja que as alterações sejam aplicadas.

Git

Comandos Básicos

- **Checkout**: ferramenta versátil usada para navegar entre ramos (ramificações), restaurar arquivos e inspecionar estados anteriores de um repositório
  - Trocando para outro ramo: **main**

```
git checkout main          ou    git switch main
```

  - Criando e trocando para um novo ramo **etapa3**

```
git checkout -b etapa3     ou    git restore etapa3
```

  - Restaurando arquivos (descartando mudanças)

```
git checkout -- nome_arquivo
```

Git

Comandos Básicos

- **Merge**: faz a mesclagem de arquivos de dois ramos. Ele é usado para combinar alterações de um ramo com as de outro ramo, integrando o histórico das duas ramificações, e criando um novo **commit** de mesclagem

```
git merge nome_arquivo_alterado
```

- **Log**: Exibe uma lista cronológica do histórico de *commits*:

```
git log
```

Git

Comandos Básicos

- **Rm**: Remove arquivos do diretório de trabalho e prepara a remoção para o próximo *commit*

```
git rm nome_arquivo
```

- **Mv: R**enomeia ou move arquivos em seu diretório de trabalho

```
git mv arquivo1.txt arquivo2.txt

git mv arquivo1.txt nova_pasta
```

- **Diff**: Compara as alterações entre a pasta de trabalho e o ***commit*** mais recente, ou entre *commits*

```
git diff nome_arquivo

git diff commit1 commit2
```

Git

Comandos Básicos

- **Config**: configura vários aspectos do **Git**, incluindo informações e preferências do usuário

```
git config --global user.email "email@servidor.com"
```

Git

Comandos Avançados

Comandos Intermediários

Comandos Básicos

```
branch, checkout, clone, merge, rebase,
tag,  push, pull, fetch, remote
```

Git

Comandos Intermediários

- **Branch**: Use para gerenciar ramificações em seu repositório Git
  - Para listar todas as ramificações existentes:

```
git branch
```

  - Para criar um ramo (*branch*) chamado “etapa2”:

```
git branch etapa2
```

  - Para renomear um ramo:

```
git branch -m nome_ramo novo_nome_ramo
```

Git

Comandos Intermediários

- **Checkout**: Alterna entre ramificações e restaura arquivos de diferentes *commits*
  - Para mudar para um ramo (*branch*) existente:

```
git checkout nome_ramo
```

  - Para descartar alterações em um arquivo específico e revertê-lo para o último *commit*:

```
git checkout --nome_arquivo
```

Git

Comandos Intermediários

- **Merge**: Para mesclar uma ramificação de recurso ou tópico no ramo principal (**main**) do Git

```
git merge nome_ramo
```

- **Rebase**: Usado para aplicar alterações de um ramo em outro, movendo ou combinando *commits*. Ele ajuda a manter um histórico de *commits* mais limpo

```
git rebase main
```

- Tag: Marca pontos específicos em seu histórico do Git, como v1.0 ou v2.0:

```
git tag v1.0
```

Git

Comandos Intermediários

- Comandos para Repositório Remoto
  - Eles conectam o repositório local a plataformas externas (GitHub, GitLab, …). Eles permitem compartilhar seu trabalho, colaborar com outras pessoas e sincronizar alterações entre diferentes ambientes.
  - Para criar uma cópia de um repositório remoto em seu computador local:

```
git clone
```

  - Para enviar os *commits* do ramo local do Git para um repositório remoto, atualizando-o com suas alterações mais recentes:

```
git push [ramo_remoto] [ramo_local]
```

Git

Comandos Intermediários

- Comandos para Repositório Remoto
  - **Pull**: Para obter e integrar as alterações de um repositório remoto em seu ramo local atual

```
git pull
```

  - **Fetch**: Para recuperar novos commits de um repositório remoto sem mesclá-los automaticamente em seu branch atual

```
git fetch origem
```

Git

Comandos Intermediários

- Comandos para Repositório Remoto
  - **Remote**: Para gerenciar repositórios remotos associados ao repositório local.
  - Para listar o repositório remoto configurado:

```
git remote
```

  - Para adicionar um novo repositório remoto, especifique seu nome e URL

```
git remote add origem https://github.com/username/origem.git
```

Git

Comandos Avançados

Comandos Intermediários

Comandos Básicos

`reset, stash`

Git

Comandos Avançados

- Os comandos Git avançados oferecem mais controle sobre o histórico do repositório e o fluxo de trabalho
  - **Reset**: Para desfazer alterações e manipular o histórico de *commits*

```
git reset nome_arquivo
```

  - Para armazenar alterações temporárias que ainda não estão prontas para receber o *commit*

```
git stash
```

  - Para listar os armazenamentos temporários e para aplicar a alteração mais recente e removê-la da lista de alterações temporárias:

```
git stash pop
```

Resumo dos Comandos

Create a Repository

From scratch -- Create a new local repository
git init [project name]

Download from an existing repository
git clone my_url

Observe your Repository

List new or modified files not yet committed
git status

Show the changes to files not yet staged
git diff

Show the changes to staged files
git diff --cached

Show all staged and unstaged file changes
git diff HEAD

Show the changes between two commit ids
git diff commit1 commit2

Working with Branches

List all local branches
git branch

List all branches, local and remote
git branch -av

Switch to a branch, my_branch, and update working directory
git checkout my_branch

Create a new branch called new_branch
git branch new_branch

Delete the branch called my_branch
git branch -d my_branch

Merge branch_a into branch_b
git checkout branch_b
git merge branch_a

Tag the current commit
git tag my_tag

Resumo dos Comandos

List the change dates and authors for a file
git blame [file]

Show the file changes for a commit id and/or file
git show [commit]:[file]

Show full change history
git log

Show change history for file/directory including diffs
git log -p [file/directory]

Make a change

Stages the file, ready for commit
git add [file]

Stage all changed files, ready for commit
git add .

Commit all staged files to versioned history
git commit -m "commit message"

Commit all your tracked files to versioned history
git commit -am "commit message"

Unstages file, keeping the file changes
git reset [file]

Revert everything to the last commit
git reset --hard

Resumo dos Comandos

Synchronize

Get the latest changes from origin (no merge)
git fetch

Fetch the latest changes from origin and merge
git pull

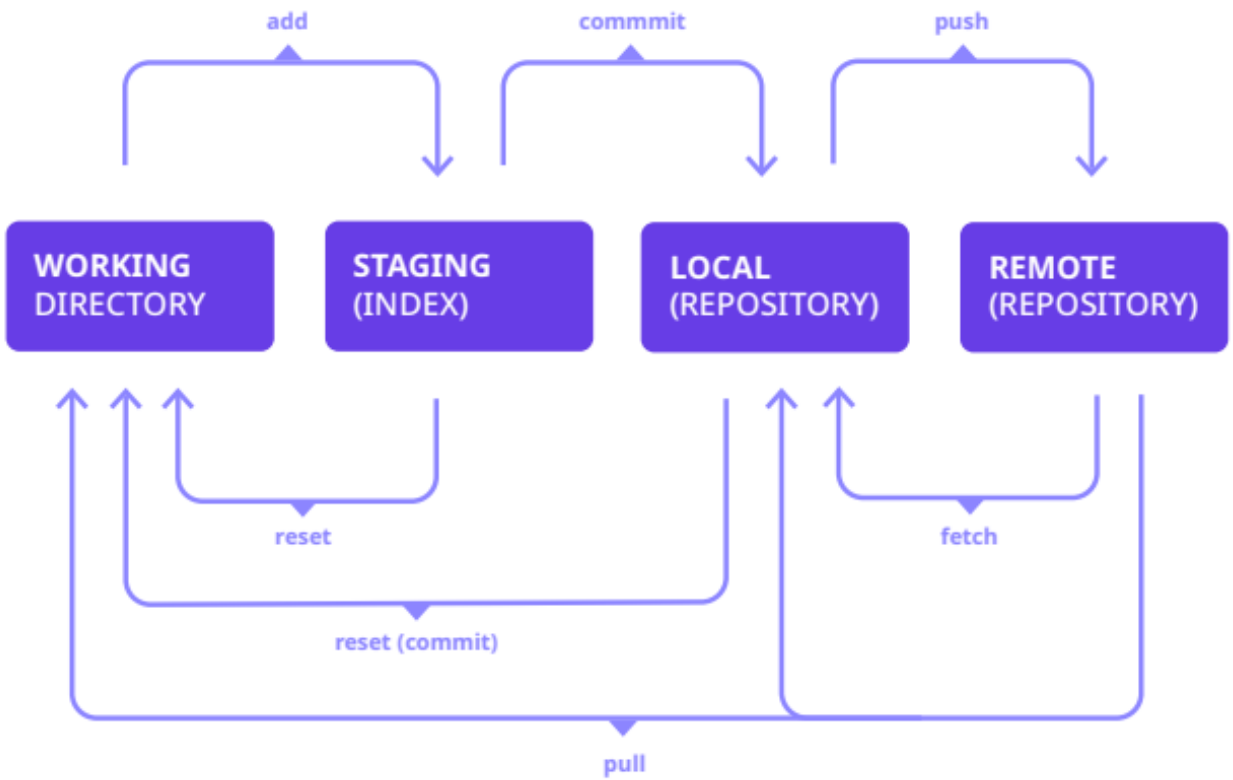
Fetch the latest changes from origin and rebase
git pull --rebase

Push local changes to the origin
git push

Finally!

When in doubt, use git help
git command --help

Or visit **https://training.github.com/** for official GitHub training.



Guias de Consulta Rápida

Instale o Git

GitHub fornece clientes desktop que incluem uma interface gráfica para as ações mais comuns em um repositório e atualiza automaticamente para a linha de comando do Git para cenários avançados.

GitHub para Windows

windows.github.com

GitHub para Mac

mac.github.com

Distribuições do Git para Linux e sistemas POSIX são disponíveis no site oficial do Git SCM.

Git para todas as plataformas

git-scm.com

Guias de Consulta Rápida

Configure a ferramenta

Configure informações de usuário para todos os repositórios locais

```
$ git config --global user.name "[nome]"
```

Configura o nome que você quer ligado às suas transações de commit

```
$ git config --global user.email "[endereco-de-email]"
```

Configura o email que você quer ligado às suas transações de commit

Crie repositórios

Inicie um novo repositório ou obtenha de uma URL existente

```
$ git init [nome-do-projeto]
```

Cria um novo repositório local com um nome especificado

```
$ git clone [url]
```

Baixa um projeto e seu histórico de versão inteiro

Refatore nomes de arquivos

Mude e remova os arquivos versionados

```
$ git rm [arquivo]
```

Remove o arquivo do diretório de trabalho e o prepara a remoção

```
$ git rm --cached [arquivo]
```

Remove o arquivo do controle de versão mas preserva o arquivo localmente

```
$ git mv [arquivo-original] [arquivo-renomeado]
```

Muda o nome do arquivo e o prepara para o commit

Suprima o monitoramento

Ignore arquivos e diretórios temporários

```
*.log
build/
temp-*
```

Um arquivo de texto chamado `.gitignore` suprime o versionamento acidental de arquivos e diretórios correspondentes aos padrões especificados

```
$ git ls-files --others --ignored --exclude-standard
```

Lista todos os arquivos ignorados neste projeto

Guias de Consulta Rápida

Salve fragmentos

Arquive e restaure mudanças incompletas

```
$ git stash
```

Armazena temporariamente todos os arquivos monitorados modificados

```
$ git stash pop
```

Restaura os arquivos recentes em stash

```
$ git stash list
```

Lista todos os conjuntos de alterações em stash

```
$ git stash drop
```

Descarta os conjuntos de alterações mais recentes em stash

Faça mudanças

Revise edições e crie uma transação de commit

`$ git status`

Lista todos os arquivos novos ou modificados para serem commitados

`$ git diff`

Mostra diferenças no arquivo que ainda não foram preparadas

`$ git add [arquivo]`

Faz o snapshot de um arquivo na preparação para versionamento

`$ git diff --staged`

Mostra a diferença entre arquivos preparados e suas últimas versões

`$ git reset [arquivo]`

Retira o arquivo da área de preparação, mas preserva seu conteúdo

`$ git commit -m "[mensagem descritiva]"`

Grava o snapshot permanentemente no histórico de versão

Mudanças em grupo

Nomeie uma série de commits e combine os esforços completos

`$ git branch`

Lista todos os branches locais no repositório atual

`$ git branch [nome-do-branch]`

Cria um novo branch

`$ git switch -c [nome-do-branch]`

Muda para o branch especificado e atualiza o diretório de trabalho

`$ git merge [nome-do-branch]`

Combina o histórico do branch especificado ao branch atual

`$ git branch -d [nome-do-branch]`

Exclui o branch especificado

Guias de Consulta Rápida

Revise o histórico

Navegue e inspecione a evolução dos arquivos do projeto

```
$ git log
```

Lista o histórico de versões para o branch atual

```
$ git log --follow [arquivo]
```

Lista o histórico de versões para um arquivo, incluindo mudanças de nome

```
$ git diff [primerio-branch]...[segundo-branch]
```

Mostra a diferença de conteúdo entre dois branches

```
$ git show [commit]
```

Retorna mudanças de metadata e conteúdo para o commit especificado

Guias de Consulta Rápida

Desfaça commits

Apague enganos e crie um histórico substituto

`$ git reset [commit]`

Desfaz todos os commits depois de `[commit]`, preservando mudanças locais

`$ git reset --hard [commit]`

Descarta todo histórico e mudanças para o commit especificado

Sincronize mudanças

Registre um repositório remoto e troque o histórico de versão

`$ git fetch [nome-remoto]`

Baixe todo o histórico de um repositório remoto

`$ git merge [nome-remoto]/[branch]`

Combina o branch remoto ao branch local atual

`$ git push [alias] [branch]`

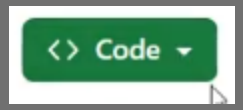
Envia todos os commits do branch local para o GitHub

`$ git pull`

Baixa o histórico e incorpora as mudanças

Exemplo 1: Criação de um Repositório no Github

Usando o Github para criar um *clone* de um repositório:



- Depois de logar em sua conta, clique em **<> Code**
- Depois copie o endereço do repositório mostrado na aba **Local**

```
https://github.com/kaw19/hello-git-github
```

- Acesse a pasta da sua máquina que conterá o projeto a ter controlado as versões, e supondo que já exista o **Git Bash** instalado na sua máquina, abra uma janela Git Bash:

```
leand@DESKTOP-GL1TDFK MINGW64 /e/Code
$
```

- Agora clone o repositorio criado no Github para a sua máquina:

```
leand@DESKTOP-GL1TDFK MINGW64 /e/Code
$ git clone
Cloning into 'hello-git-github'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 6 (delta 0), reused 0 (delta 0),
Unpacking objects: 100% (6/6), done.

leand@DESKTOP-GL1TDFK MINGW64 /e/Code
$
```

Exemplo 1: Criação de um Repositório no Github

- Agora acesse a pasta do projeto na sua máquina

```
leand@DESKTOP-GL1TDFK MINGW64 /e/Code
$ cd hello-git-github/

leand@DESKTOP-GL1TDFK MINGW64 /e/Code/hello-git-github (main)
$
```

- A pasta possui apenas o arquivo README.md criado pelo Github na ocasião da criação do repositório

Name	Date modified	Type	Size
README.md	24/06/2023 15:57	MD File	1 KB

E:\Code\hello-git-github\README.md - Notepad++

File Edit Search View Encoding Language Settings Tools Macro

README.md

```
1  # Hello GIT Github
2
```

Arquivo formatado em **Markdown** (linguagem de marcação leve criada para formatação de linhas de textos...)

https://markdownguide.org/getting-started/

Exemplo 2: Criando e usando uma Ramificação

Para criar um *branch* use:

```
git branch nome_ramo_novo
```

Para nos movermos para o *branch* criado:

```
git checkout nome_ramo_novo
```

Agora, no novo *branch*, podemos criar quantas modificações no código quisermos sem ter que mudar nada na versão principal.

Para listar novamente as ramificações, veremos que um novo *branch* foi adicionado e que estamos localizados nele:

```
git branch
```

Precisamos fazer um ***commit*** para o *branch* principal para o Git entender qual é o *branch* **master** (principal), antes de criarmos um *branch* novo…

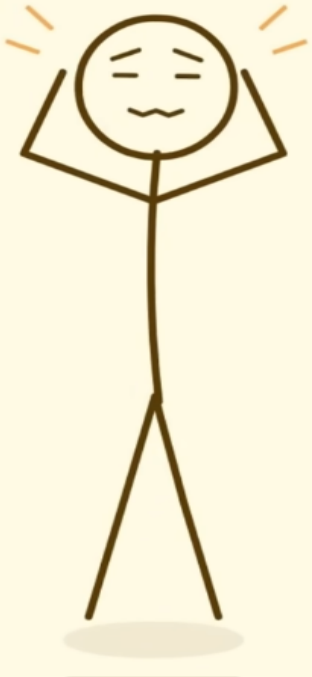
```
git checkout master
git branch -d nome_ramo_novo
```

Git

Erros Comuns

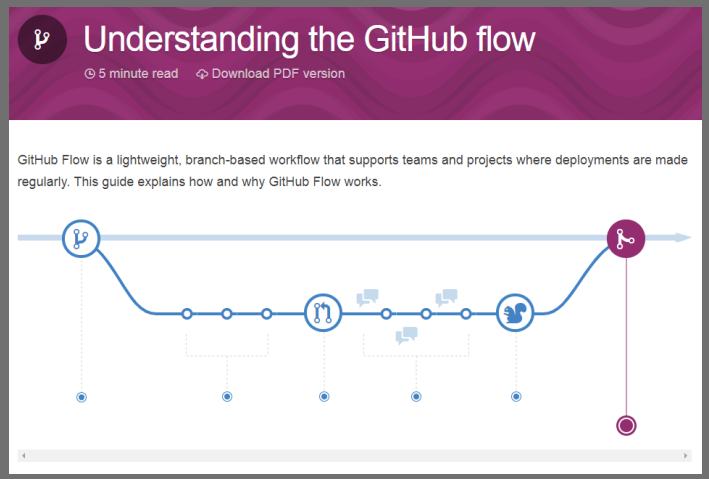
```
$ git push
error: failed to push some refs to
'origin/main'
hint: Updates were rejected because the remote
contains work
hint: that you do not have locally.

fatal: refusing to merge unrelated histories
```



Fontes

- Bruno Santana; “Comandos Git: uma lista dos mais usados para simplificar seu trabalho”; 09/01/26; https://www.hostinger.com/br/tutoriais/comandos-git; acesso em 17/04/26.
- Andrei L.; “Como usar um Git branch”; 04/08/23; disponível em https://www.hostinger.com/br/tutoriais/git-branch; acesso em 17/04/26.
- Site; “The Official GitHub Training Manual”; https://githubtraining.github.io/training-manual/#/
- Yan Min Thwin; “Understanding GitHub Flow and Git Flow”; 23/07/23; disponível em https://medium.com/@yanminthwin/understanding-github-flow-and-git-flow-957bc6e12220; acesso em 22/04/26.



O **GitHub Flow** é um fluxo de trabalho leve e baseado em ramificações, projetado para equipes que implantam código com frequência. Sua filosofia central é que a ramificação principal esteja sempre estável e pronta para implantação.